

# Sistemas Operacionais

---

## Memória Virtual

# Conteúdo

---

- Memória Virtual
- Paginação sob demanda
- Desempenho da paginação sob demanda
- Algoritmos de substituição de páginas
- Alocação de Frames
- Thrashing
- O modelo working set

# MEMÓRIA VIRTUAL

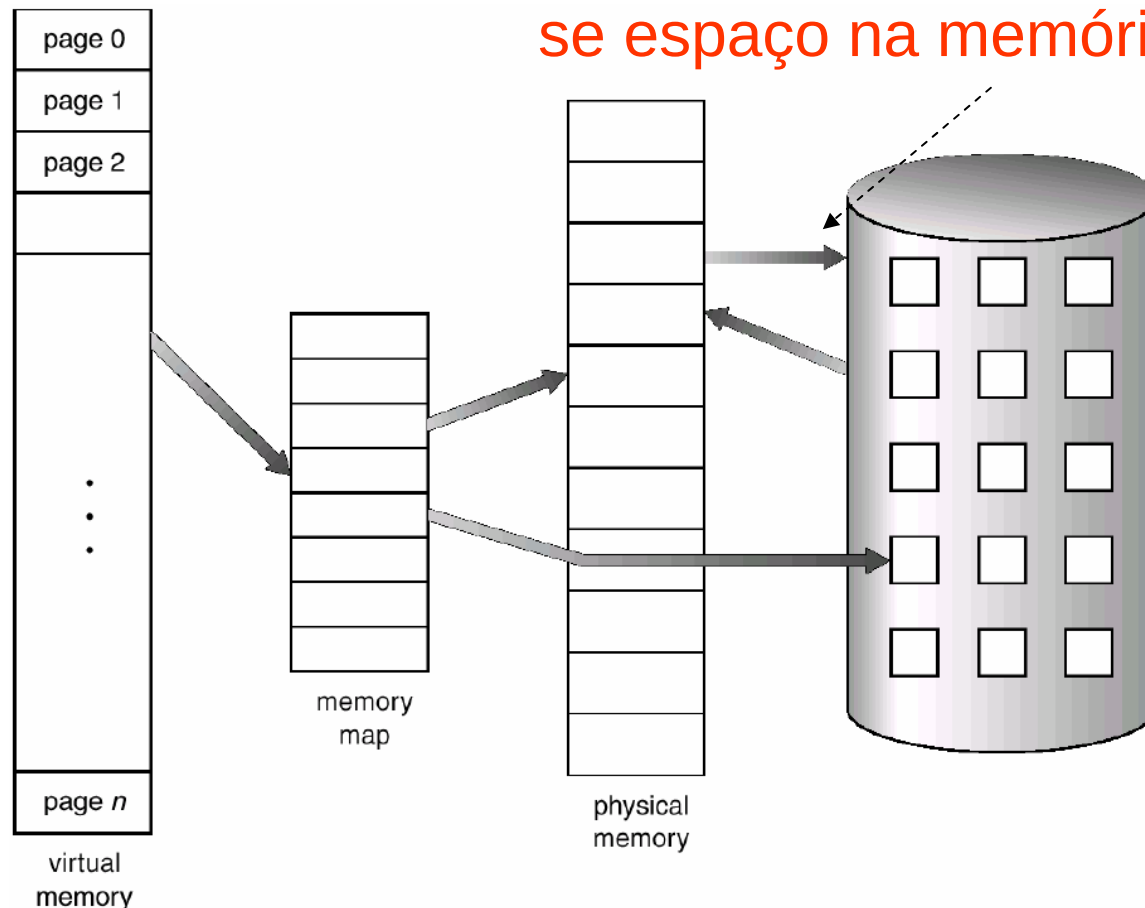
---

- Em geral, somente parte do programa precisa estar na memória principal para ser executado
- Permite → faixa de end. Lógico >> faixa end. físico
- End. lógicos são mapeados dinamicamente para end. físicos
- Endereços podem ser compartilhados por vários processos
- Criação de processos podem ser mais eficientes
- Mais processos podem ser mantidos na memória simultaneamente → melhor utilização da **CPU**
- Um processo pode ser maior do que a memória física → usuário não precisa se preocupar com isso

# Memória virtual (cont.)

- Páginas que não estejam na memória física são trazidas da memória secundária

Páginas podem ser salvas no HD  
se espaço na memória física estiver limitado

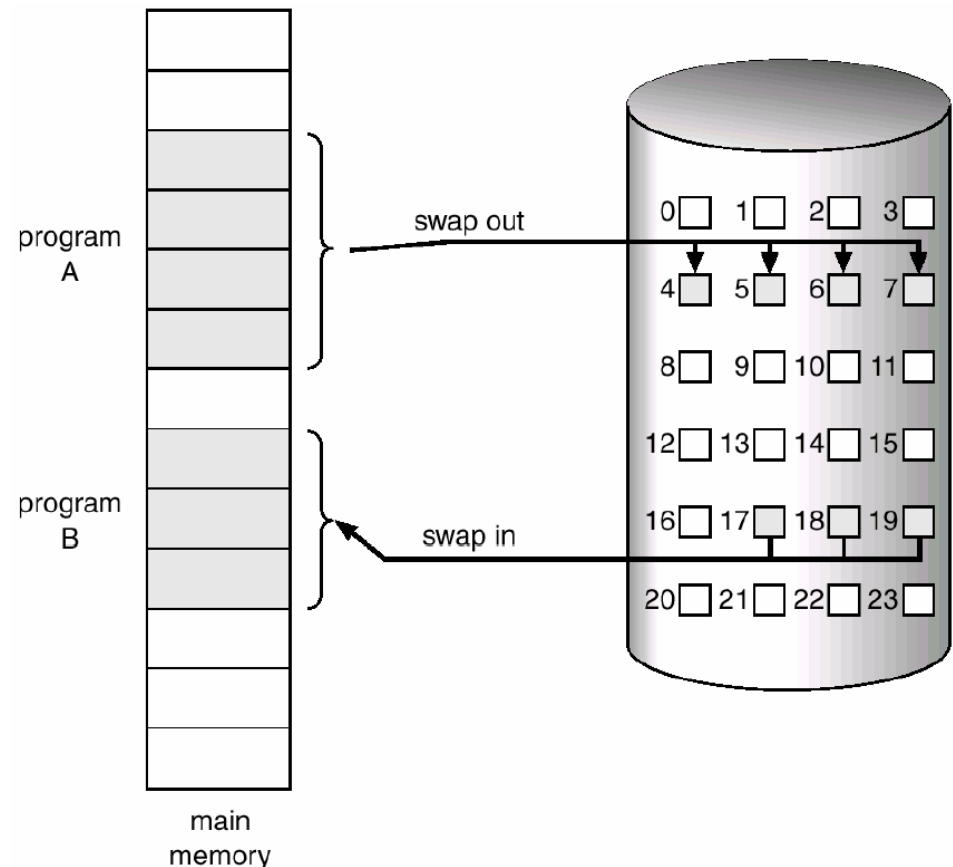


## Desafios:

- 1- qdo trocar uma página?
- 2- qual página substituir?
- 3- qdo não é preciso salvar página sobrescrita no HD?
- 4- ...

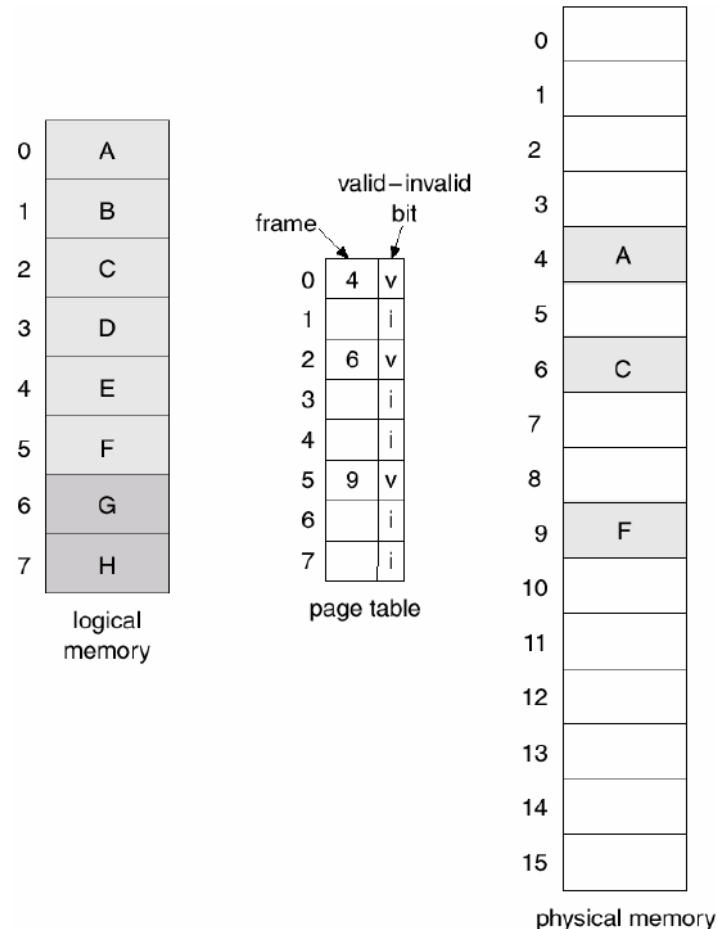
# Paginação sob demanda

- Páginas só são trazidas para a memória principal (física) quando necessárias
- Páginas referenciadas:
  - Ausente em memória  
→ carregar
  - Inválida  
→ abortar por erro
- Referência a uma página ausente, gera uma *system call* do tipo ***page fault***



# Paginação sob demanda (cont.)

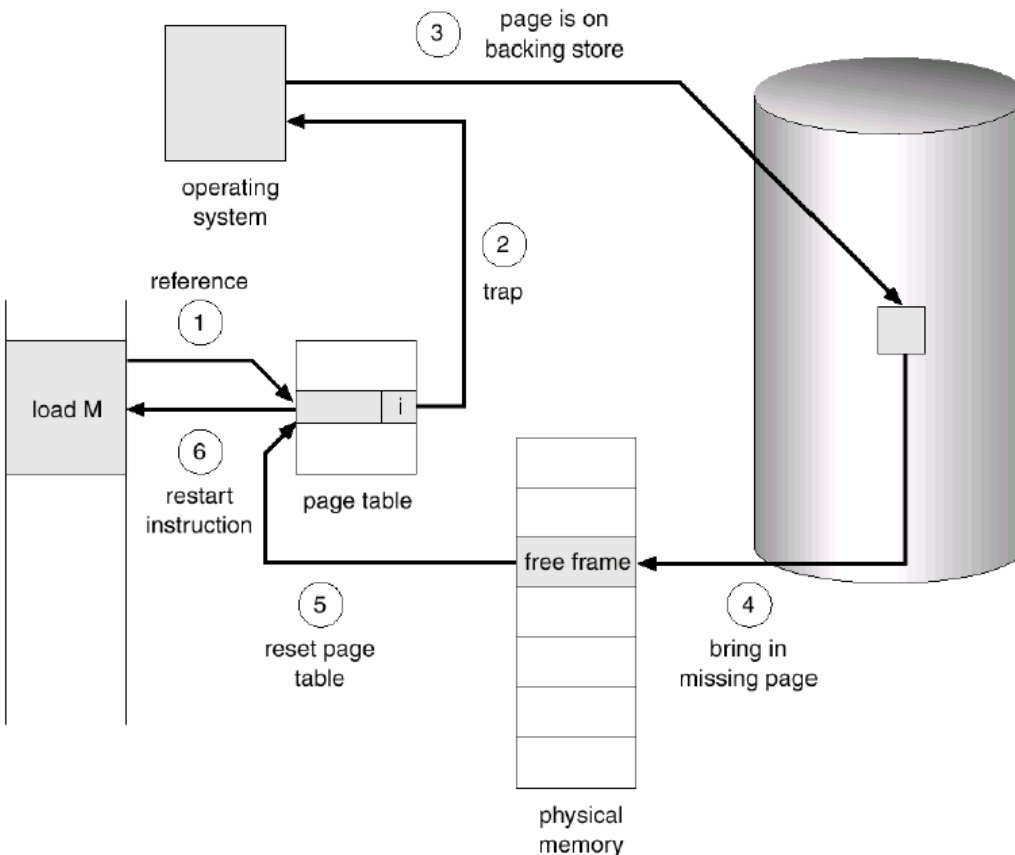
- Cada registro da tabela de páginas inclui o bit de validade “v”:
  - 1 (valid) → em memória, 0 (invalid) → no disco
- Referência a um registro com  $v=0$  (invalid) gera uma **page fault**
- Inicialmente todos os registros são ajustados com  $v=0$



# Procedimento sob *page fault*

1- qdo uma página não é encontrada na memória principal ( $v=0$ ), as seguintes tarefas ocorrem:

- 2- uma system call (trap) invoca o SO que determina se o endereço recebido é válido. Se não, aborta o processo
- 3- um frame livre é encontrado
- 4- página é trazida p/ o frame
- 5- tabelas são atualizadas,  $v=1$
- 6- instrução que desencadeou esse procedimento é reiniciada



# Desempenho da paginação sob demanda

- O impacto da paginação sob demanda no desempenho do sistema pode ser substancial
- Na ausência de **page faults**, o tempo de acesso efetivo de memória (TAEM) paginada é igual ao tempo de acesso à memória (TAM), o qual varia entre 100 e 200 ns
- Com uma probabilidade de **page faults**  $p$  ( $0 < p < 1$ ), o TAEM fica:

$$\text{TAEM} = (1-p) \times \text{TAM} + p \times \text{TPF}$$

Tempo de  
Page Fault

Fatores que influem o RFP:

- tratamento da interrupção de page fault
- leitura da página do disco
- reinício do processo

# Desempenho da pag. sob demanda (cont.)

## EXEMPLO

- Um HD típico tem os seguintes atrasos médios:
  - Latência: 8 ms, busca: 15 ms, transferência: 1 ms
- Somando-se aos tempos de tratamento da interrupção de **page fault** e reinício da instr, tem-se um total de cerca de 25 ms com **page fault** (TPF)
- Então, com TPF=25 ms e TAM=100 ns:  
$$TAEM = (1-p) \times 100 \text{ ns} + p \times (25 \text{ ms}) = 100 + 24999900 \times p$$
- Para se ter menos de 10% de degradação, precisa-se:  
$$110 > 100 + 25000000 \times p \rightarrow 10 > 25000000 \times p \rightarrow$$
  
$$\rightarrow p < 0.0000004$$

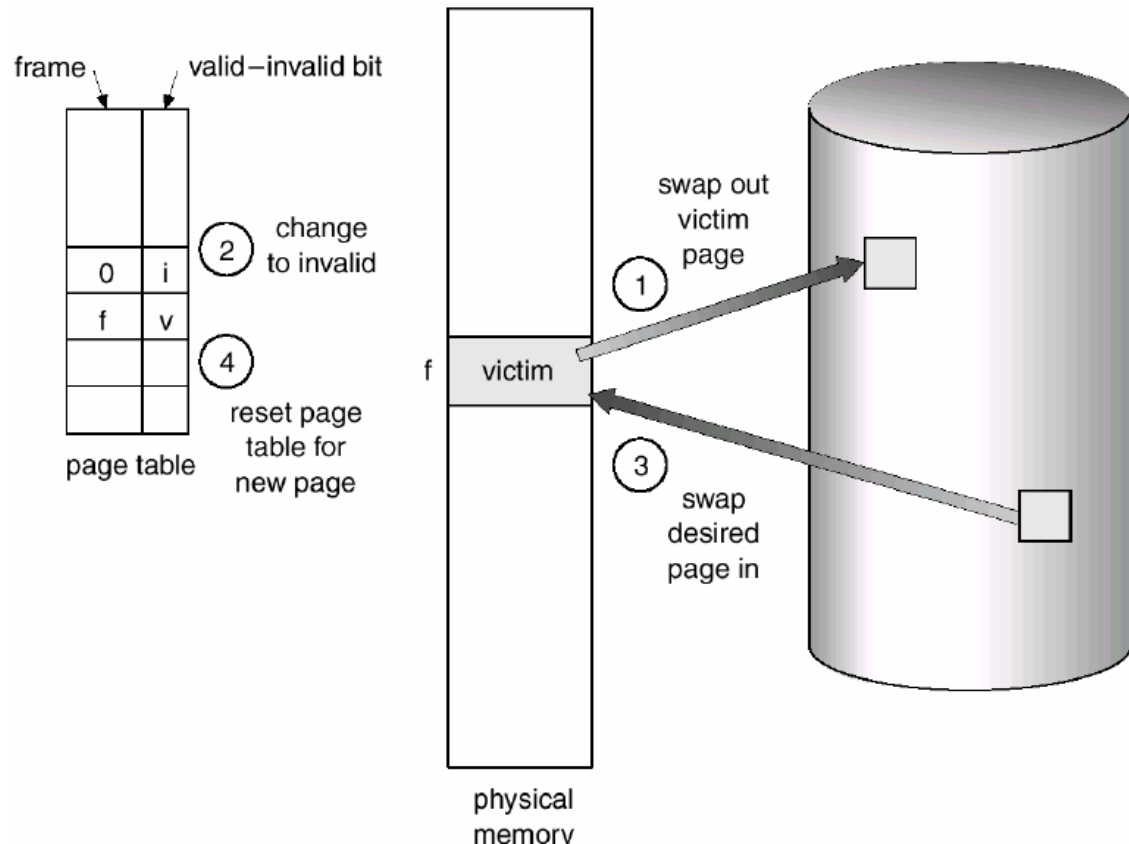
Menos de uma **page fault** dentre 2500000 acessos à memória !!!

# Substituição de páginas

- Qdo ocorre **page fault** com necessidade de busca de uma página do disco e não há frames livres
  - Um frame vítima deve ser copiado para o disco para liberar espaço
- Assim, uma extensa memória virtual pode funcionar com uma memória física menor
- O mecanismo que lida com **page faults** deve incluir o algoritmo de substituição mostrado

modify (ou dirty) bit reduz  
tempo de **page fault**

→ Páginas não alteradas,  
não copiar p/ o disco



# Algoritmos de substituição de páginas

---

- Objetivam a menor taxa de *page faults* possível
- Para avaliá-los, necessita-se de
  - uma cadeia de seqüência de referências à memória
  - conhecer a quantidade de frames disponíveis
- Na maioria dos exemplos que se seguem será utilizada a seguinte cadeia de seqüência:
  - 1 2 3 4 1 2 5 1 2 3 4 5
- Portanto, a página 1 é referenciada primeiramente, depois a página 2, e assim por diante

# Algoritmo de substituição ideal (optimal)

- Usado como padrão de referência para outros algoritmos, pois um algoritmo perfeito não existe
- Com este algoritmo
  - Obtém-se a menor taxa de **page faults**
  - Nunca ocorrerá a anomalia de Belady (vista mais adiante)
- Idéia→ substitua a página que demorará mais a ser usada
- Exemplo de **4 frames**

cadeia de referência: 1 2 3 4 1 2 5 1 2 3 4 5

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 5 |
| - | 1 | 2 | 3 | 3 | 4 |
| - | - | 1 | 2 | 2 | 2 |
| - | - | - | 1 | 1 | 1 |

**6 page faults**

- Algoritmo de substituição ideal é difícil de ser implementado porque requer conhecimento prévio da sequência de **page faults**

# Algoritmo de substituição FIFO

- Muito simples de ser implementado

## EXEMPLOS:

cadeia de referência: 1 2 3 4 1 2 5 1 2 3 4 5

- **3 frames** (3 págs podem estar na memória por processo)

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 1 | 2 | 5 | 3 | 4 |
| - | 1 | 2 | 3 | 4 | 1 | 2 | 5 | 3 |
| - | - | 1 | 2 | 3 | 4 | 1 | 2 | 5 |

*9 page faults*

- **4 frames**

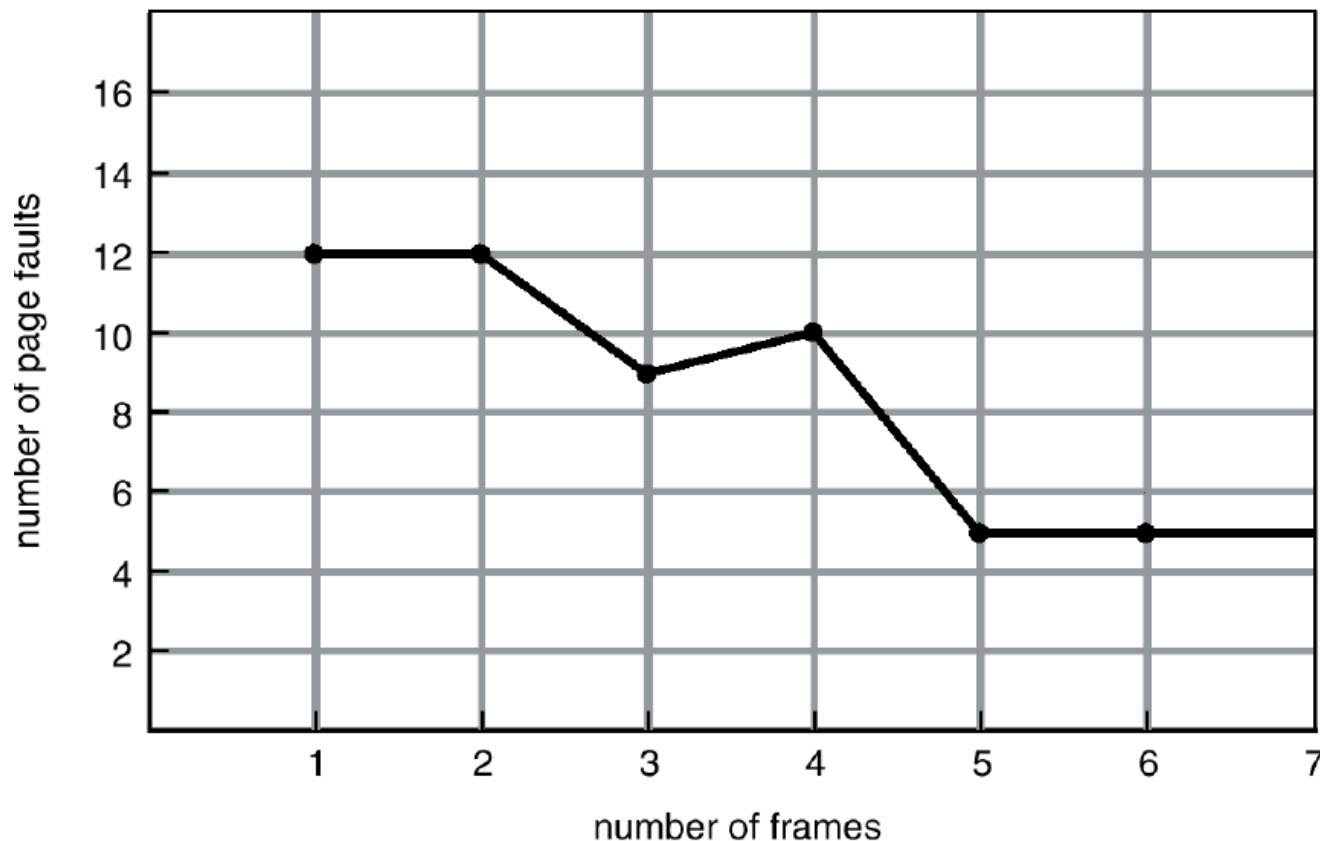
|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 1 | 2 | 3 | 4 | 5 |
| - | 1 | 2 | 3 | 4 | 5 | 1 | 2 | 3 | 4 |
| - | - | 1 | 2 | 3 | 4 | 5 | 1 | 2 | 3 |
| - | - | - | 1 | 2 | 3 | 4 | 5 | 1 | 2 |

*10 page faults*

- Comportamento inesperado:
  - mais frames livres → mais *page fault* → **Anomalia de Belady**

# Algoritmo FIFO com anomalia de Belady

- Gráfico mostra que com 4 frames livres tem-se mais **page faults** (10) do que com 3 frames livres (9)

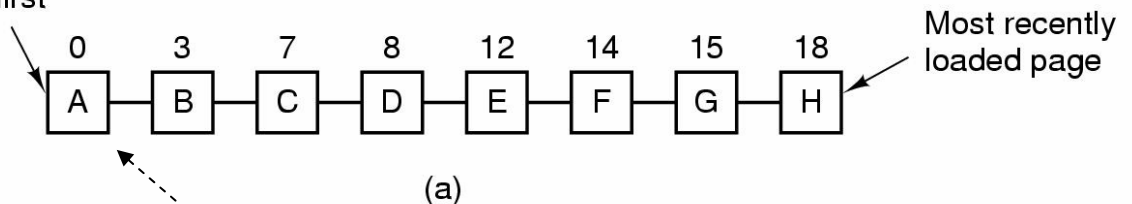


# Algoritmo de substituição Segunda Chance

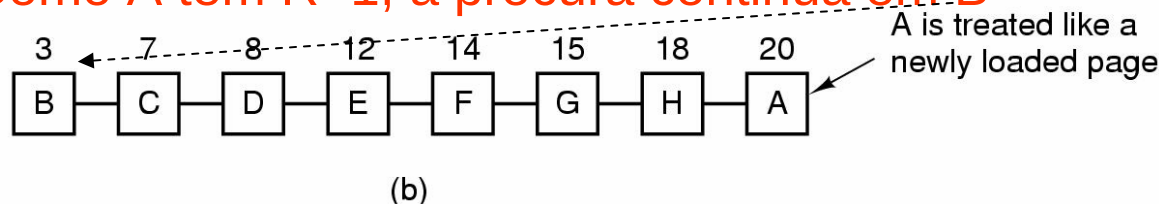
- Pequena melhora ao algoritmo FIFO
- Registros da tabela de páginas incluem *bit de referência R*
- Página referenciada  $\rightarrow R=1$
- A fila FIFO é organizada com base no tempo de carga de cada página
- $R=0 \rightarrow$  página velha e não usada  $\rightarrow$  subst. imediatamente
- $R=1 \rightarrow R$  é colocado em 0, a página vai para o fim da fila, e o seu tempo de carga é atualizado com se a mesma tivesse acabado de ter sido carregada

Este algoritmo evita o problema de se substituir páginas que estão sendo muito usadas !

Page loaded first

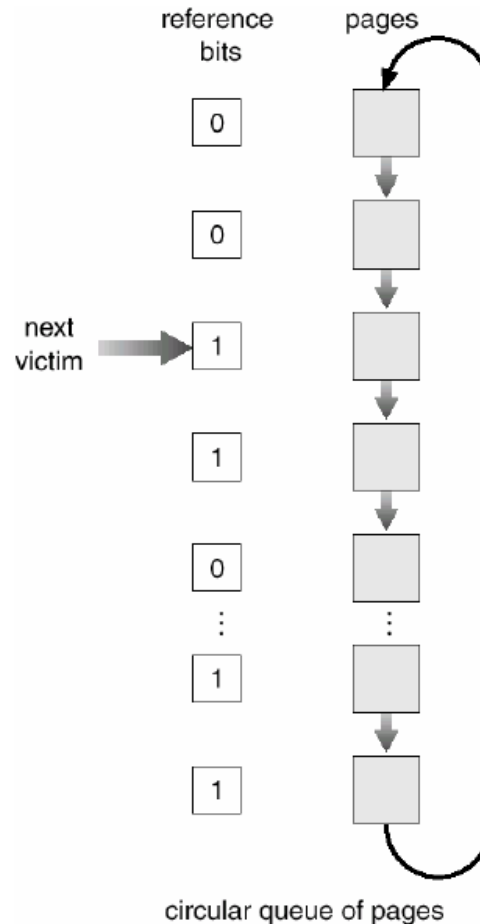


Como A tem  $R=1$ , a procura continua em B

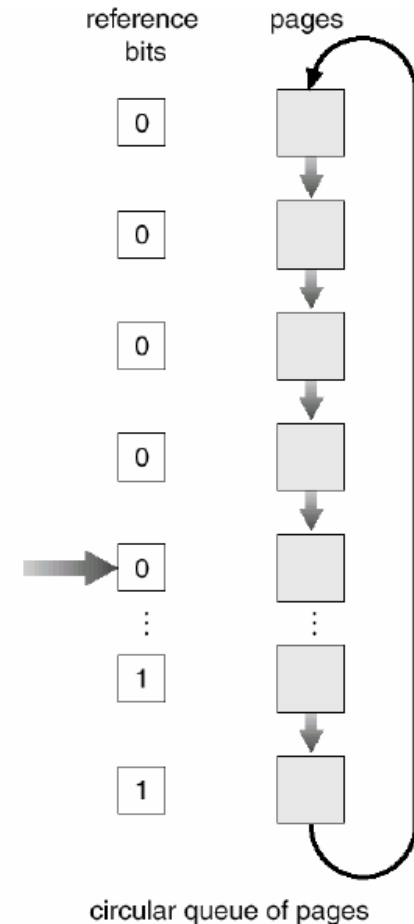


# Algoritmo de substituição Clock

- É uma implementação particular do algorit. Segunda Chance
- Diferentemente do algoritmo Segunda Chance, não precisa deslocar páginas dentro da fila
- Páginas a serem substituídas são pesquisadas por ordem circular
- Se  $R=1$ , então
  - Fazer  $R=0$
  - Não subst. página
  - Ir para próx. página



(a)



(b)

Se todas as páginas estiverem com  $R=1$ , o algoritmo fará uma busca completa até retornar à página inicial

# Algoritmo de substituição LRU

- LRU (Least Recently Used): menos usado recentemente
- LRU representa uma aproximação do algoritmo ideal
  - Substitua a página não usada a mais tempo
- Associa-se a cada página, o tempo de seu último uso
- Exemplo de **4 frames**

cadeia de referência: 1 2 3 4 1 2 5 1 2 3 4 5

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 4 | 3 | 3 | 3 |
| - | 1 | 2 | 3 | 5 | 5 | 4 | 4 |
| - | - | 1 | 2 | 2 | 2 | 2 | 2 |
| - | - | - | 1 | 1 | 1 | 1 | 5 |

*8 page faults*

- Este algoritmo é bastante usado, mas normalmente demanda substancial suporte do hardware

# Comparação entre alguns dos algoritmos

- Neste exemplo, o LRU é o algoritmo que mais se aproxima do algoritmo ideal

Page address  
stream

2 3 2 1 5 2 4 5 3 2 5 2

OPT

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 2 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 2 | 2 | 2 |
|   | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
|   |   |   | 1 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
|   |   |   |   | F |   | F |   |   | F |   |   |

3 page faults

LRU

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 3 | 3 | 3 |
|   | 3 | 3 | 3 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
|   |   |   | 1 | 1 | 1 | 4 | 4 | 4 | 2 | 2 | 2 |
|   |   |   |   | F |   | F |   | F | F |   |   |

4 page faults

FIFO

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 2 | 2 | 2 | 5 | 5 | 5 | 5 | 3 | 3 | 3 | 3 |
|   | 3 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 5 | 5 |
|   |   |   | 1 | 1 | 1 | 4 | 4 | 4 | 4 | 4 | 2 |
|   |   |   |   | F | F | F |   | F |   | F | F |

6 page faults

CLOCK

|    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|
| 2* | 2* | 2* | 2* | 5* | 5* | 5* | 5* | 3* | 3* | 3* | 3* |
|    | 3* | 3* | 3* | 3  | 2* | 2* | 2* | 2  | 2* | 2  | 2* |
|    |    |    | 1* | 1  | 1  | 4* | 4* | 4  | 4  | 5* | 5* |
|    |    |    |    | F  | F  | F  |    | F  |    | F  |    |

5 page faults

\* indica R=1

# Exercício

---

- Considere a seguinte cadeia de referência de páginas  
1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6
- Quantas **page faults** ocorrerão com os algoritmos de substituição de páginas IDEAL, FIFO e LRU?
- Considere duas situações distintas, com 3 e 5 frames.

# Alocação de Frames

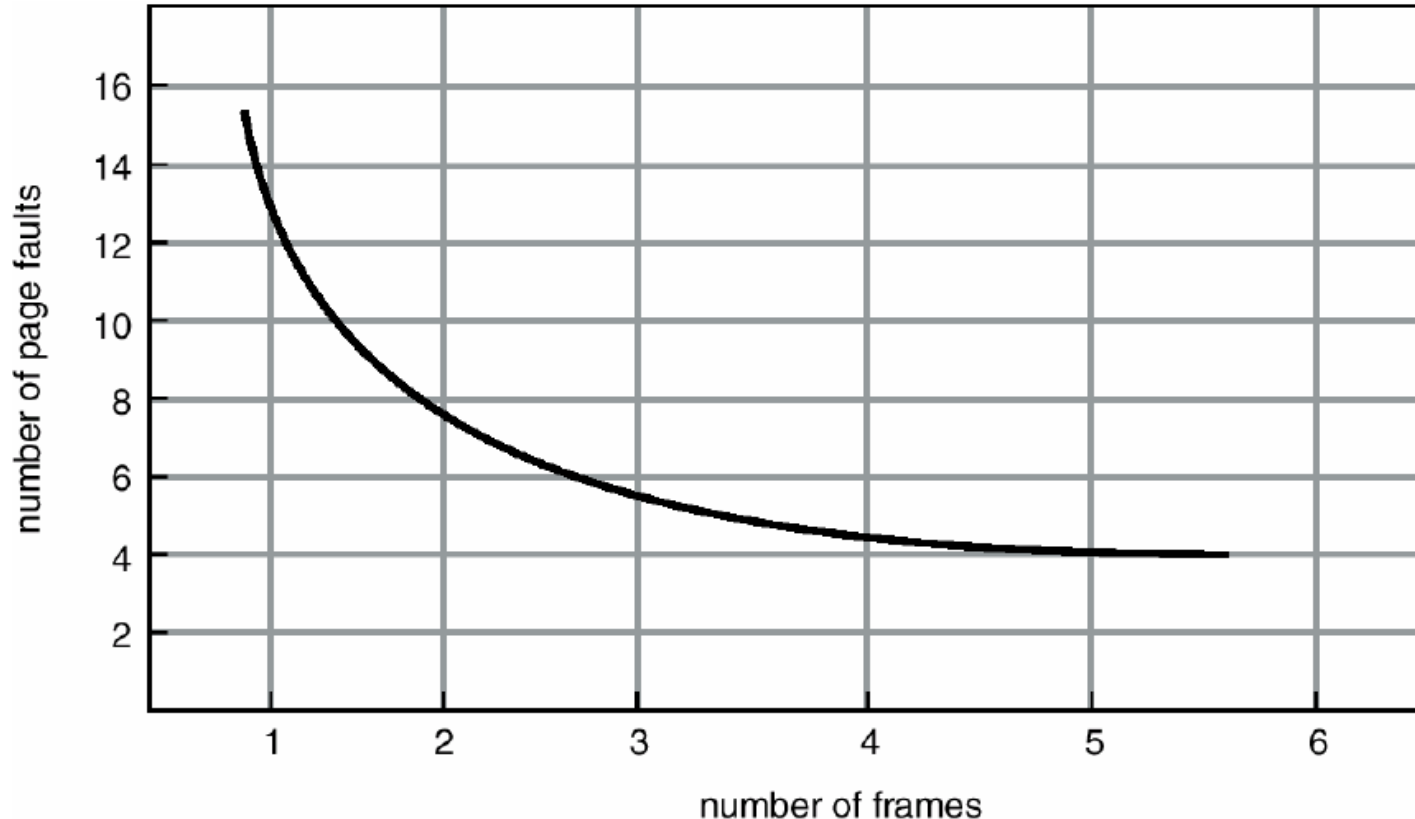
---

- Cada processo precisa de um número mínimo de frames livres
  - No mínimo alguns poucos frames para uma instrução completa
  - Depende da arquitetura de instrução do processador em uso
- Mínimo de frames → depende da arquitetura
- Máx. de frames → função da memória física disponível
- Maneira mais fácil p/ alocação de frames é dividir os frames disponíveis igualmente entre os processos
  - Problema: processos têm necessidades distintas
- Uma solução alternativa → alocação proporcional
- Uma terceira possibilidade é a alocação por prioridade

# Alocação de Frames (cont.)

- Comportamento esperado:

A medida que o número de frames disponíveis aumenta, a taxa de ***page fault*** torna-se constante e pequena



# Alocação Local vs. Alocação Global

---

**Na substituição dos frames, duas possibilidades são possíveis**

## Substituição Local

- Cada processo seleciona frames somente do seu próprio conjunto de frames alocados

## Substituição Global

- Processos selecionam frames do conjunto total de frames alocados. Assim, processos podem pegar frames uns dos outros

# Thrashing

- Se um processo não tem páginas o suficiente, a taxa de **page faults** é muito alta. Isso ocasiona
  - Baixa utilização de CPU
  - SO adiciona outro processo ao sistema → thrashing

## O princípio da localidade

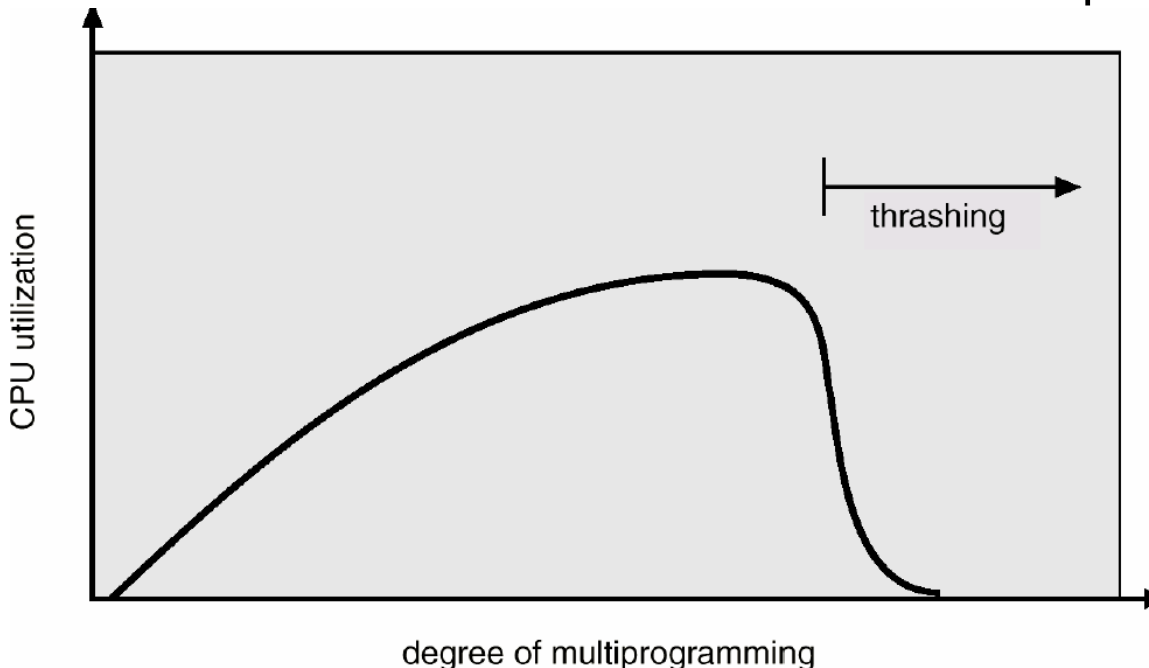
programas e dados de um processo tendem a ficar localizados em intervalos de tempo

frames alocados < localidade atual

## Thrashing

um processo está mais ocupado com swaps do que com trabalho efetivo

Uma mesma página entra e sai da memória repetidamente

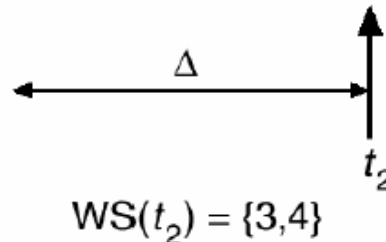
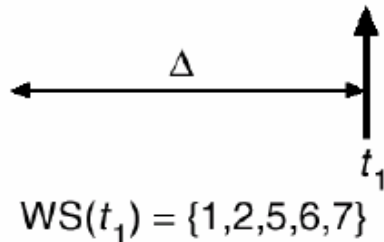


# O modelo Working Set

- O working set é uma aproximação da localidade de um processo
- A idéia é manter na memória, um número mínimo de páginas que satisfa a localidade do momento
- Definindo-se um intervalo de tempo delta, o working set  $WS_i$  de um processo no instante  $t_i$  compõe-se das páginas referenciadas no último intervalo delta
- Páginas do working set não devem ser substituídas

page reference table

... 2 6 1 5 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



**O working set**

- 1- evita thrashing
- 2- otimiza uso da CPU

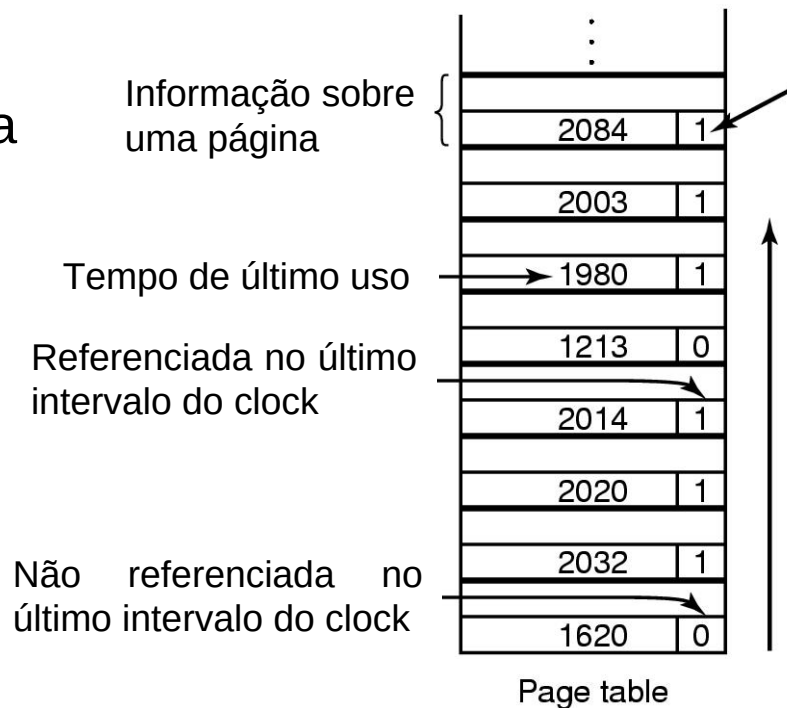
# Implementação do modelo Working Set

- Normalmente usa-se um temporizador mais um bit de referência (R) para controle do working set
- Em intervalos regulares, o working set é inspecionado

**tempo efetivo de uso da CPU** -----> 2204 Tempo virtual corrente

Cada registro da tabela contém:

- Bit de referência
- Tempo do último uso da página



R: bit de referência

**A cada pulso do clock → R=0**

verifica o bit R de todas as páginas  
se (R==1)

Tempo de último uso ← tempo virtual corrente

se (R==0 e idade > delta)

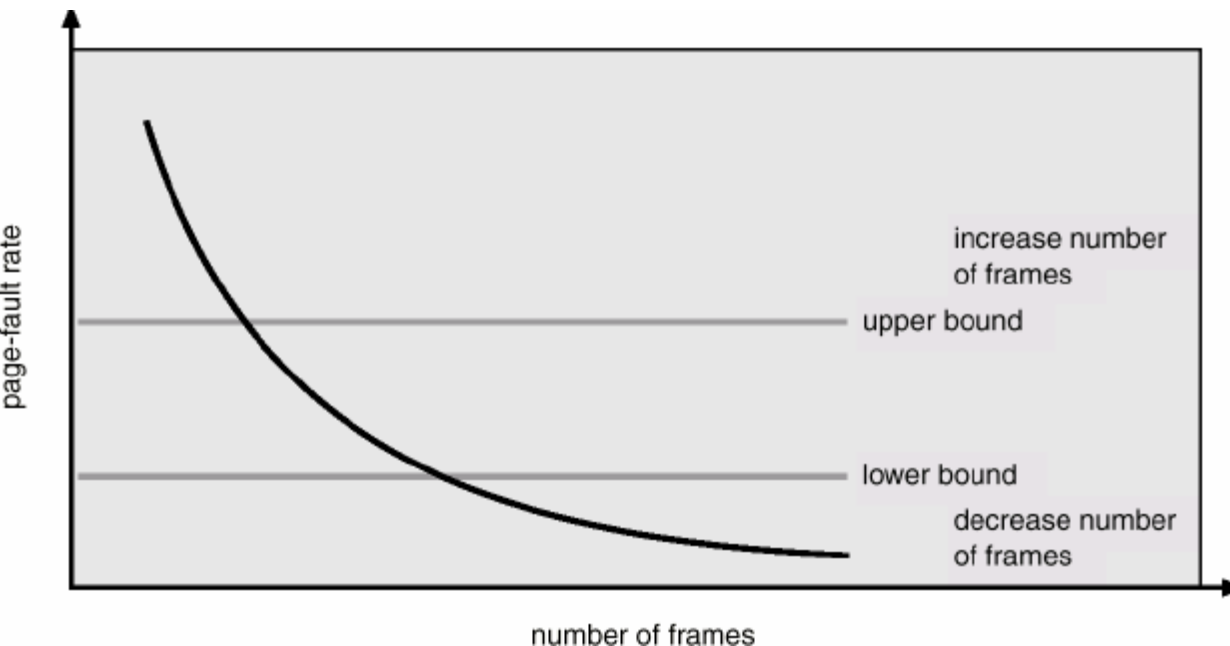
remova a página

se (R==0 e idade ≤ delta)  
anote tempo de último uso da página

**Seqüência sob page fault**

# Esquema de frequência de *page faults*

- Controla o thrashing monitorando a frequência de *page faults*
- Mais fácil de se implementar do que o working set
- Define-se uma faixa aceitável de taxas de *page faults*
- Acima do limite aloca mais frame ao processo, e libera frames se taxa cai abaixo do limite



Na falta de frames  
livres a serem  
alocados, suspender  
um processo e  
liberar seus frames

# Leitura Complementar

---

- ***Sistemas Operacionais Modernos, 2ª edição***  
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**  
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***  
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**  
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004